

KFB Travel v7 — Architektur-Audit

Das Fundament für KFB Travel v7 ist hervorragend strukturiert. Die konsequente Trennung von **Bewegung (Controller)**, **Präsentation (CardRig)** und **Pet-Verhalten (PetKinetics)** beweist, dass das Projekt die Prototyp-Phase verlassen hat und zu einer robusten Engine heranreift. Dieser Report konsolidiert den Status Quo und liefert den klaren Fahrplan (Handover) für die Zusammenarbeit am v8-Branch.

1. Status Quo & Risikoanalyse

Die aktuellen Module zeigen starke Ownership-Grenzen, was für die Skalierung des Systems entscheidend ist. Der Controller kümmert sich ausschließlich um das Movement, während PetKinetics als sauberer Layer für sekundäre Reaktionen aufgesetzt ist.

Kritischer Flaschenhals: `travel-poc.js` (ca. 62 KB) übernimmt aktuell zu viele Aufgaben (Bootstrap, Orchestrierung, Render-Loop, Controller-Wechsel). Es besteht akute Gefahr, dass dieses Skript zu einem God Object mutiert. Dies muss vor der Implementierung weiterer Features gestoppt werden.

2. Architektur-Roadmap für Version 8

Die folgenden Refactorings haben höchste Priorität (P1), um das System für neue Fahrzeuge und Umgebungen fit zu machen:

A. Einführung eines TravelManager

Der TravelManager übernimmt die Rolle des Orchestrators. `travel-poc.js` wird dadurch auf reines Bootstrapping reduziert. Der Manager kümmert sich um Lifecycle, Update-Pipeline und Modus-Wechsel.

B. Explizite Controller-Verträge (Interfaces)

Bisherige implizite Verträge müssen hart definiert werden, damit bei neuen Controllern (Hover, Boot, Ballon) nicht von der Architektur abgewichen wird.

```
interface ITravelController { enter(state); exit(state); update(dt, input, worldContext); getState(); // Gibt immutable State zurück }
```

C. Isolierung des CameraRig

Die Kamera darf niemals Fahrzeug-Logik auslesen oder manipulieren. Das CameraRig muss ein reiner Konsument des Runtime-States werden (nimmt Speed, Heading, TurnRate entgegen und berechnet daraus Lag, FOV und Shake).

3. Design & Art-Direction Sync (Design Handover)

Damit das digitale Game euer klassisches analoges Cut-and-Play Comic-Gefühl optimal transportiert, muss die Code-Architektur diesen retro-analogen, handgezeichneten Stil technisch stützen, ohne die Physik zu kompromittieren:

- **Animations-Layering im CardRig:** Die Trennung von reiner Physik (Controller) und Visualisierung (CardRig) ist hier der Schlüssel. Während die Engine intern flüssig und linear rechnet, sollte das CardRig State-Trigger für State-Machines oder Sprite-Animationen bereitstellen. So kann das visuelle Feedback dynamisch und unperfekt im Sinne klassischer analoger Line-Art bleiben, ohne das Gameplay schwammig zu machen.
- **PetKinetics als Resonanz-Körper:** Das Pet reagiert ausschließlich auf den TravelState (Squash & Stretch, Ear Lag). Diese Trennung unbedingt beibehalten! Sie verhindert, dass Gameplay-Code mit UI/Character-Animation verschmutzt wird.
- **WorldContext als reine Lese-Instanz:** Umgebungsabfragen bleiben strikt getrennt von Render-Logik.

4. Coworker Checkliste (Sprint v8)

Prüfpunkt	Beschreibung / Regel
Ownership	Keine Präsentations- oder UI-Logik in den Controllern.
Pipeline	Strikte Update-Reihenfolge einhalten: Input → WorldContext → TravelController → Vehicle → PetKinetics → CardRig → FX → Camera → Render.
Camera	Kamera ist Konsument, kein Akteur. Sie liest nur den State.
Event Bus	Zentralisiertes Event-System (Mount, Dismount, Boost) einführen.

Fazit: Die Code-Basis ist bereit für den Übergang vom Prototypen zur vollwertigen Engine. Die formale Festlegung von Interfaces und die Entschlackung der POC-Datei sind die einzigen Blockaden für den nächsten großen Meilenstein.